KONGU ENGINEERING COLLEGE

INDUSTRIAL DIESEL GENERATOR MONITORING AND PREVENTIVE MAINTENANCE SYSTEM

MECHATRONICS AND IOT

EXCERSICE - 04

HEMAVARSHINI S - 24MER017

KAVIN KUMARAN S - 24MER023

KOWSIKA K - 24MER026

VISHNUPRASATH B - 24MER062

KONGU ENGINEERING COLLEGE

TRANSFORM YOURSELF

INDUSTRIAL DIESEL GENERATOR MONITORING AND PREVENTIVE MAINTENANCE SYSTEM

1. Project Overview

Introduction

Diesel generators are widely used in industries as backup and continuous power sources. Conventional monitoring of these generators often relies on manual inspection at fixed intervals, which can allow abnormal operating conditions to go unnoticed until a failure occurs.

The Industrial Diesel Generator Monitoring and Preventive Maintenance System is an IoT-based automated monitoring system designed to continuously observe the operating condition of a diesel generator. The system uses an ESP32 microcontroller as the main controller, along with a voltage sensor, an ACS712 current sensor, and a DHT11 temperature and humidity sensor, to acquire the important operating parameters of the generator in real time.

The measured parameters are displayed locally through the Serial Monitor and simultaneously transmitted to the ThingSpeak IoT cloud platform over Wi-Fi. A buzzer alarm is incorporated to alert the operator immediately whenever an abnormal condition, such as high temperature, over-current, or abnormal voltage, is detected.

The main objective is to shift generator maintenance from a purely time-based, manual approach toward continuous condition-based monitoring, enabling early fault detection and preventive maintenance.

Main Features

- Continuous monitoring of generator voltage, current, temperature and humidity.
- Automatic threshold-based abnormal-condition detection.
- Local buzzer alarm for immediate operator alert.
- Real-time display of readings through the Serial Monitor.
- Cloud data logging and visualization through ThingSpeak.
- Wi-Fi based IoT connectivity using ESP32.
- Historical data monitoring for preventive maintenance.
- Low-cost and expandable monitoring architecture.

2. Problem Statement

Diesel generators are widely used as backup and continuous power sources in industries. Conventional generator monitoring often depends on manual inspection and periodic, time-based maintenance schedules. This approach does not take the actual operating condition of the generator into account.

This can result in:

- Delayed detection of abnormal operating conditions.

- Excessive temperature going unnoticed.
- Over-current conditions damaging the generator or connected load.
- Voltage fluctuations affecting equipment.
- Unexpected generator failure.
- Increased maintenance cost.
- Unplanned downtime and loss of production.

Therefore, a system is required to continuously monitor generator parameters and provide an early warning when abnormal conditions occur, rather than relying solely on periodic manual checks.

3. Proposed Solution

The proposed system uses sensors and a microcontroller to automate generator condition monitoring. The voltage sensor measures the generator's output voltage, the ACS712 sensor measures the current drawn by the load, and the DHT11 sensor measures the ambient temperature and humidity around the generator. All three sensor outputs are sent to the ESP32.

The ESP32 processes these values and compares them against predefined threshold limits to determine whether the generator is operating normally or abnormally.

When a parameter goes outside its normal range:

Sensor (Voltage / Current / Temperature) → ESP32 → Buzzer ON → Operator Alerted
When all parameters remain within the normal range:

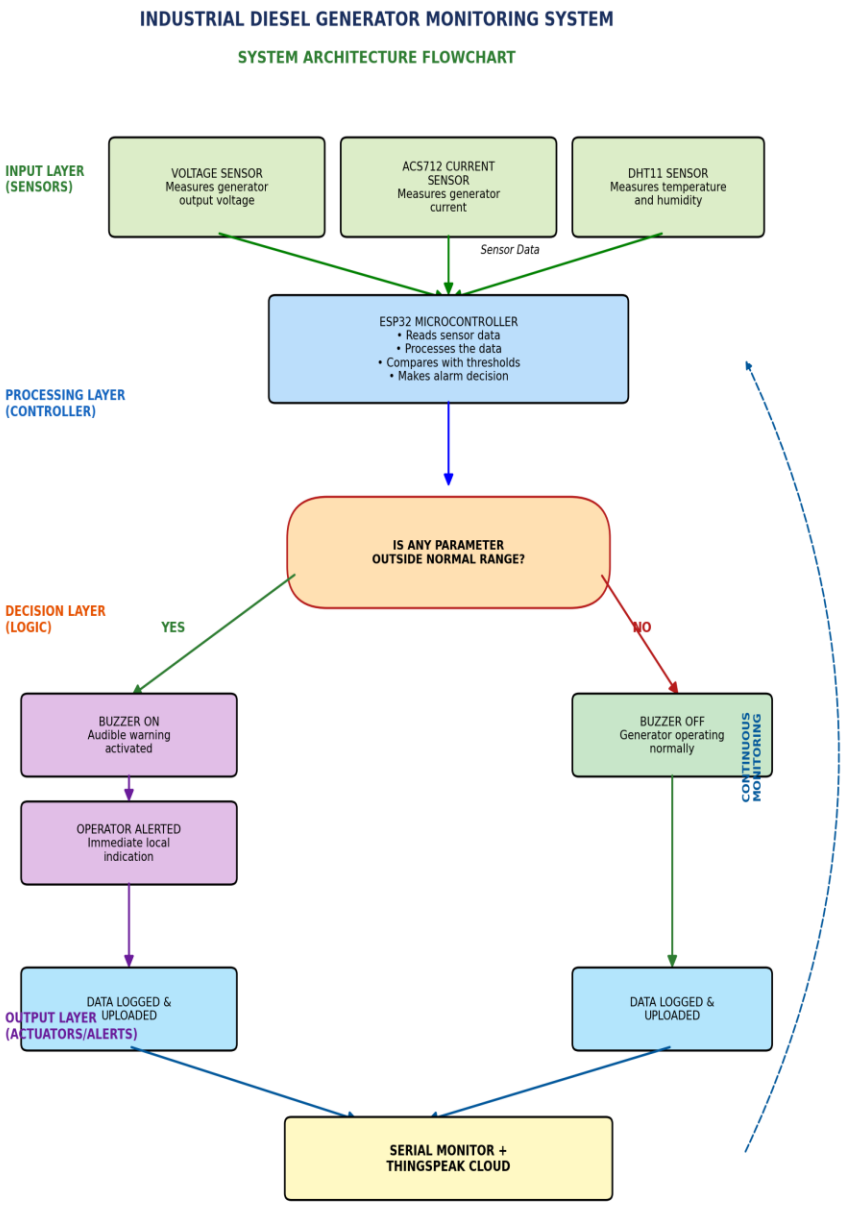
Sensor (Voltage / Current / Temperature) → ESP32 → Buzzer OFF → Generator Status: Normal
In parallel with the local alarm logic, the ESP32 continuously displays the readings on the Serial Monitor and uploads them to the ThingSpeak cloud platform, enabling both immediate local warning and remote historical monitoring for preventive maintenance.

4. System Architecture

The overall architecture of the system is:

System Flow:

Sensors → ESP32 Microcontroller → Threshold Comparison → Buzzer / Cloud Upload → Preventive Maintenance



5. Circuit Table

Component	Pin	ESP32
Voltage Sensor Module	OUT	GPIO 34
Voltage Sensor Module	VCC	3V3
Voltage Sensor Module	GND	GND
ACS712 Current Sensor	OUT	GPIO 35
ACS712 Current Sensor	VCC	5V
ACS712 Current Sensor	GND	GND

DHT11 Sensor	DATA	GPIO 4
DHT11 Sensor	VCC	3V3
DHT11 Sensor	GND	GND
Buzzer	SIG (+)	GPIO 25
Buzzer	GND (–)	GND

Power Supply

The ESP32 can be powered through USB or a regulated 5V supply. The voltage sensor module and ACS712 current sensor should be supplied according to their rated input voltage, and the sensing side must be electrically isolated/scaled to remain within the ESP32's safe analog input range. The buzzer and DHT11 operate directly from the ESP32's 3V3/5V rails.

6. Circuit Design

The circuit consists of four major sections:

6.1 ESP32 Microcontroller

The ESP32 is the main controller. It reads the analog and digital sensor signals, compares them with threshold values, and controls the buzzer accordingly. It also connects to Wi-Fi to upload data to ThingSpeak.

6.2 Voltage Sensor

The voltage sensor module is connected to analog input GPIO 34. It provides a scaled analog value representing the generator's output voltage.

Voltage Sensor OUT → ESP32 GPIO 34

6.3 ACS712 Current Sensor

The ACS712 measures the current drawn by the generator load and is connected to analog input GPIO 35.

ACS712 OUT → ESP32 GPIO 35

6.4 DHT11 Sensor

The DHT11 measures:

- Temperature.
- Relative humidity.

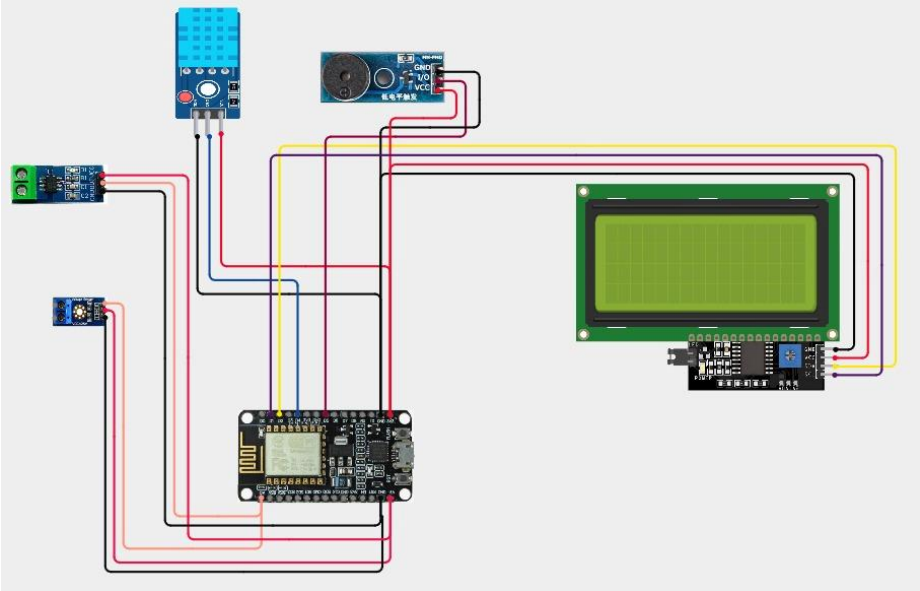
DHT11 DATA → ESP32 GPIO 4

6.5 Buzzer

The buzzer provides an immediate local audible alert. It is switched by the ESP32 whenever any monitored parameter crosses its threshold.

ESP32 GPIO 25 → Buzzer SIG

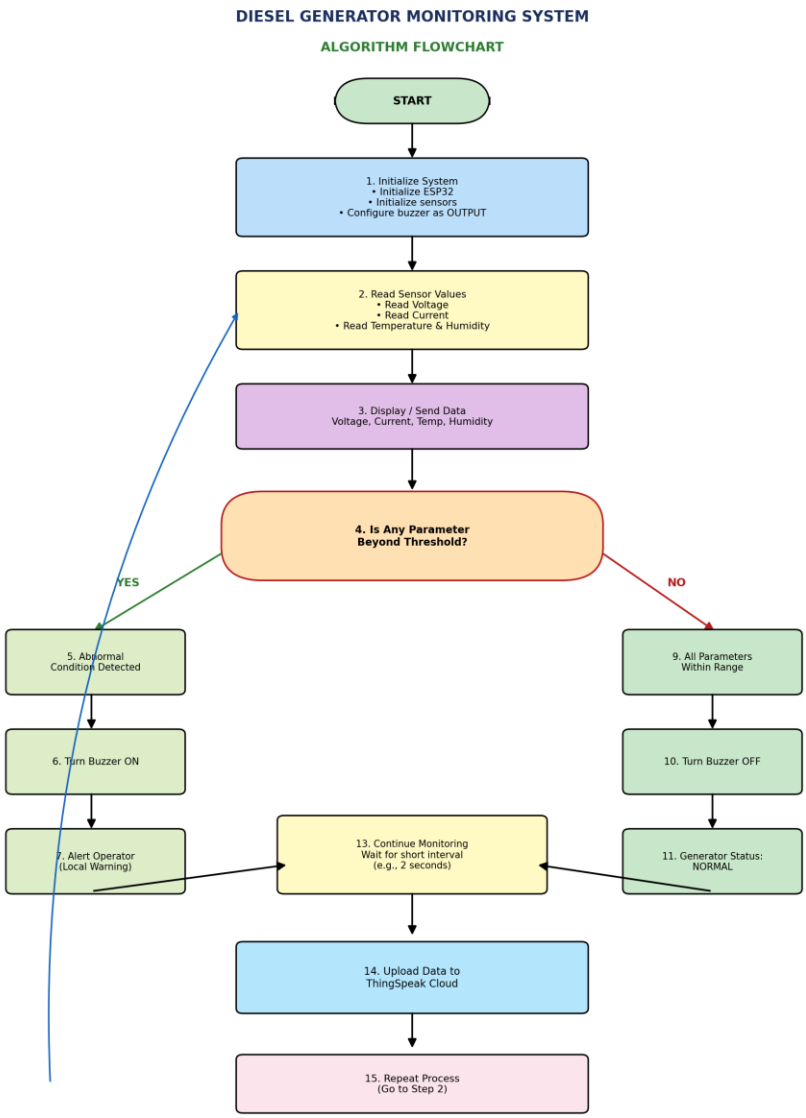
CIRCUIT DESIGN:



7. Algorithm

Algorithm

1. Start the system.
2. Initialize the ESP32 and connect to Wi-Fi.
3. Initialize the voltage sensor, ACS712 current sensor, and DHT11 sensor.
4. Configure the buzzer pin as an output.
5. Read the voltage sensor value.
6. Read the current sensor value.
7. Read temperature and humidity from the DHT11.
8. Compare the measured parameters with predefined threshold values.
9. If any parameter is abnormal (over-voltage, over-current, or high temperature), turn ON the buzzer.
10. If all parameters are within the normal range, turn OFF the buzzer.
11. Display the readings and generator status on the Serial Monitor.
12. Upload the sensor data to the ThingSpeak cloud platform.
13. Wait for a short interval.
14. Repeat the process continuously.



8. Coding

The following Arduino program is designed for the ESP32 used in the project.

```
#include <WiFi.h>

#include <HTTPClient.h>

#include <DHT.h>



const char* ssid = "kowsi";
```

```
const char* password = "kowsi @ 01";

const char* thingspeakAPI =
  "J2CV7BDPTKMTOEJC";

const char* server =
  "http://api.thingspeak.com/update";

#define BUZZER_PIN     25
#define CURRENT_PIN    35
#define DHT_PIN        4
#define VOLTAGE_PIN    34

#define DHT_TYPE DHT11

DHT dht(DHT_PIN, DHT_TYPE);

// ACS712-5A  = 0.185 V/A
// ACS712-20A = 0.100 V/A
// ACS712-30A = 0.066 V/A

// Change according to your ACS712
float ACS_SENSITIVITY = 0.100;

// Voltage sensor calibration factor
float VOLTAGE_FACTOR = 25.0;

// ESP32 ADC
float ADC_VOLTAGE = 3.3;
```

```
void setup()
{
  Serial.begin(115200);

  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, LOW);

  dht.begin();

  analogReadResolution(12);

  Serial.println();
  Serial.println("==========================================");
  Serial.println("    DIESEL GENERATOR MONITORING SYSTEM");
  Serial.println("==========================================");

  WiFi.begin(ssid, password);

  Serial.print("Connecting to WiFi");

  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("WiFi Connected!");

  Serial.print("ESP32 IP Address: ");
```

```
  Serial.println(WiFi.localIP());

  Serial.println("-----------------------------------------");
  Serial.println("ThingSpeak Cloud Monitoring Started");
  Serial.println("-----------------------------------------");

  delay(2000);
}

void loop()
{
  float temperature = dht.readTemperature();
  float humidity = dht.readHumidity();
  int currentRaw = analogRead(CURRENT_PIN);

  float currentVoltage =
    (currentRaw / 4095.0) * ADC_VOLTAGE;

  float current =
    (currentVoltage - 2.5) / ACS_SENSITIVITY;

  // Remove small negative values
  if (current < 0)
  {
    current = 0;
  }


  int voltageRaw = analogRead(VOLTAGE_PIN);
```

```
float sensorVoltage =
  (voltageRaw / 4095.0) * ADC_VOLTAGE;

float generatorVoltage =
  sensorVoltage * VOLTAGE_FACTOR;

Serial.println();
Serial.println("============================================");
Serial.println("       DIESEL GENERATOR STATUS");
Serial.println("============================================");

if (isnan(temperature))
{
  Serial.println("Temperature : Sensor Error");
}
else
{
  Serial.print("Temperature : ");
  Serial.print(temperature);
  Serial.println(" °C");
}

if (isnan(humidity))
{
  Serial.println("Humidity    : Sensor Error");
}
else
{
  Serial.print("Humidity    : ");
  Serial.print(humidity);
```

```
  Serial.println(" %");
}

Serial.print("Current     : ");
Serial.print(current);
Serial.println(" A");

Serial.print("Voltage     : ");
Serial.print(generatorVoltage);
Serial.println(" V");

bool alarm = false;

Serial.print("Generator   : ");

if (generatorVoltage < 100)
{
  Serial.println("OFF / LOW VOLTAGE");
}
else if (generatorVoltage < 180)
{
  Serial.println("WARNING - LOW VOLTAGE");
}
else if (generatorVoltage <= 250)
{
  Serial.println("NORMAL");
}
else
{
  Serial.println("WARNING - HIGH VOLTAGE");
```

```
  alarm = true;
}

if (!isnan(temperature) && temperature > 50)
{
  Serial.println("ALARM       : HIGH TEMPERATURE");
  alarm = true;
}

if (current > 20)
{
  Serial.println("ALARM       : OVER CURRENT");
  alarm = true;
}

if (alarm)
{
  digitalWrite(BUZZER_PIN, HIGH);
  Serial.println("BUZZER      : ON");
}
else
{
  digitalWrite(BUZZER_PIN, LOW);
  Serial.println("BUZZER      : OFF");
}

if (WiFi.status() == WL_CONNECTED)
{
  HTTPClient http;
```

```
String url = String(server) +
             "?api_key=" + thingspeakAPI +
             "&field1=" + String(generatorVoltage, 2) +
             "&field2=" + String(current, 2) +
             "&field3=" + String(temperature, 2) +
             "&field4=" + String(humidity, 2);

Serial.println("-------------------------------------------");
Serial.println("Sending data to ThingSpeak...");

http.begin(url);

int httpResponseCode = http.GET();

if (httpResponseCode > 0)
{
  Serial.print("ThingSpeak Response: ");
  Serial.println(httpResponseCode);

  if (httpResponseCode == 200)
  {
    Serial.println("Cloud Upload: SUCCESS");
  }
}
else
{
  Serial.print("Cloud Upload Error: ");
  Serial.println(httpResponseCode);
}
```

```
  http.end();
 }
 else
 {
  Serial.println("WiFi disconnected!");
 }

 Serial.println("==========================================");

 // ThingSpeak requires at least 15 seconds
 // between channel updates on free accounts.
 delay(20000);
}
```

Code Explanation

- GPIO 34 reads the voltage sensor.
- GPIO 35 reads the ACS712 current sensor.
- GPIO 4 receives data from the DHT11.
- GPIO 25 controls the buzzer.
- highVoltageLimit, lowVoltageLimit, overCurrentLimit and highTempLimit define the abnormal-condition thresholds.
- The DHT11 provides temperature and humidity information.
- ThingSpeak.setField() and ThingSpeak.writeFields() upload the four parameters to the cloud channel.

The calibration constants (voltageCalibration, currentSensitivity) and threshold values should be adjusted according to the actual sensors and generator ratings used.

9. System Working

When the system is powered ON, the ESP32 initializes all connected sensors, configures the buzzer as an output, and connects to the Wi-Fi network.

The voltage sensor, ACS712, and DHT11 continuously provide readings of the generator's operating condition.

Normal Operating Condition

When all measured parameters are within their normal range:

Voltage / Current / Temperature Normal
↓
ESP32 detects normal condition
↓
Buzzer OFF
↓
Generator Status: NORMAL

Abnormal Operating Condition

When any parameter crosses its threshold:

Voltage / Current / Temperature Abnormal
↓
ESP32 detects abnormal condition
↓
Buzzer ON
↓
Operator Alerted

At the same time, the ESP32 uploads the current readings to ThingSpeak, allowing the readings and alarm history to be reviewed remotely. This process repeats automatically, allowing the system to maintain continuous condition monitoring of the diesel generator.

10. Bill of Materials

S.No.	Component	Quantity
1	ESP32 Development Board	1
2	Voltage Sensor Module	1
3	ACS712 Current Sensor	1
4	DHT11 Sensor	1
5	Buzzer	1
6	Breadboard	1
7	Jumper Wires	As required
8	Regulated Power Supply	1

Software Requirements

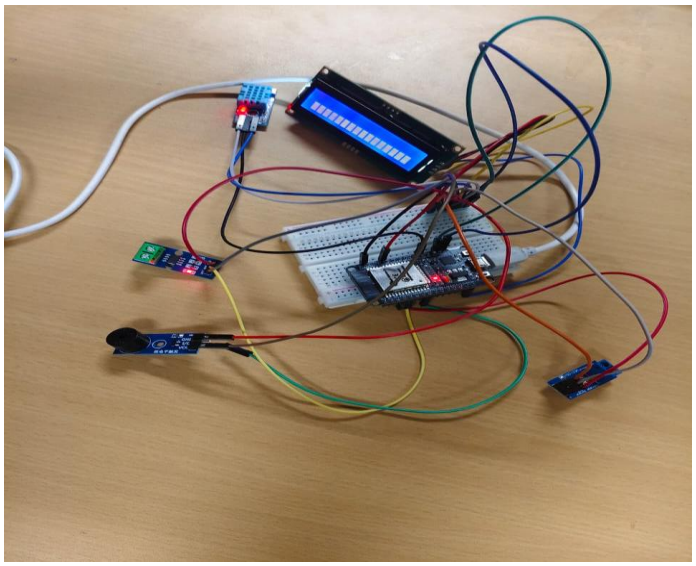
- Arduino IDE
- ESP32 Board Package
- DHT Sensor Library
- ThingSpeak Arduino Library

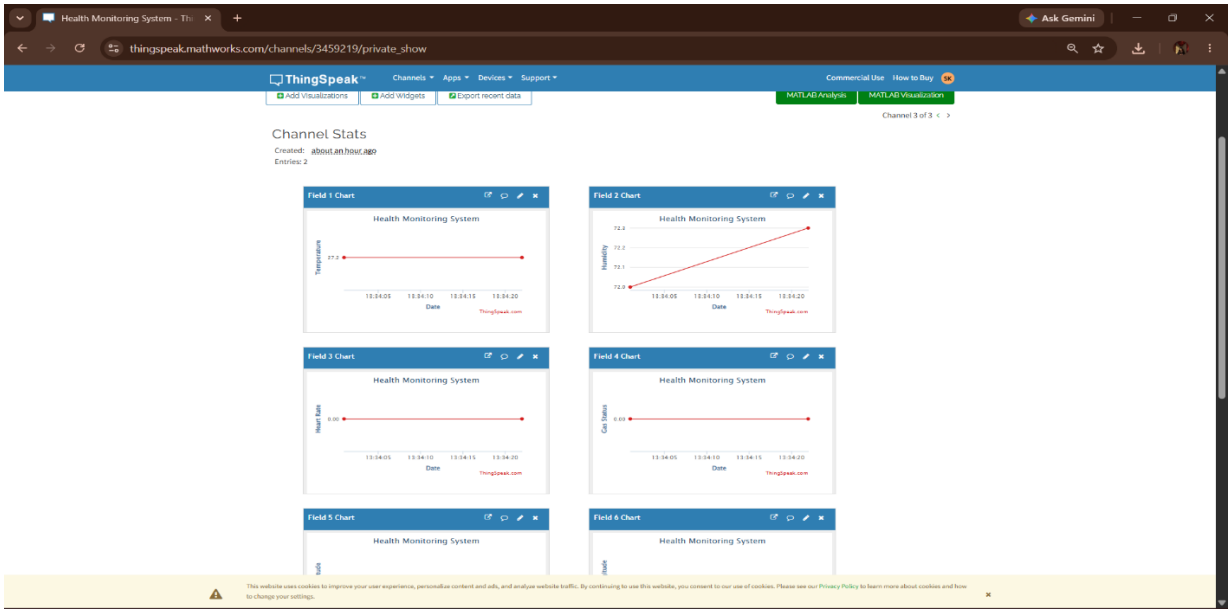
11. Prototype Implementation

The prototype was assembled using an ESP32 development board, a voltage sensor module, an ACS712 current sensor, a DHT11 sensor, and a buzzer, wired on a breadboard as per the circuit table above.

The ESP32 was selected as the main controller because it provides sufficient analog input channels, GPIO pins, and built-in Wi-Fi connectivity needed for direct IoT cloud integration, without requiring an additional communication module.

The prototype continuously reads the generator's voltage, current, temperature and humidity, compares them against the configured thresholds, drives the buzzer accordingly, and streams the readings to the Serial Monitor and the ThingSpeak channel.





12. Results & Performance Analysis

Expected Results

The system successfully demonstrates continuous condition monitoring and preventive alerting for a diesel generator.

Condition	Sensor Condition	Buzzer	Generator Status
Normal operation	All parameters within limits	OFF	NORMAL
Over-voltage / low voltage	Voltage outside set range	ON	ABNORMAL
Over-current	Current above set limit	ON	ABNORMAL
High temperature	Temperature above set limit	ON	ABNORMAL

Performance

The system provides the following benefits:

- Real-time generator monitoring.
- Early detection of abnormal operating conditions.
- Immediate local alerting through the buzzer.
- Remote monitoring and historical data logging through ThingSpeak.
- Reduced dependency on manual inspection.
- Support for condition-based preventive maintenance.
- Potential for further expansion with additional generator parameters.

Conclusion

The Industrial Diesel Generator Monitoring and Preventive Maintenance System provides a low-cost and effective approach to monitoring important generator operating parameters. By continuously measuring voltage, current, temperature and humidity, the system can detect abnormal conditions and alert the operator immediately through the buzzer, while also logging the data to the cloud for long-term trend analysis.

The ESP32 acts as the central controller, the voltage sensor and ACS712 provide the electrical parameters of the generator, and the DHT11 provides environmental information. The buzzer provides local alerting, and ThingSpeak provides remote, cloud-based monitoring.

The proposed system helps move generator maintenance from a time-based approach to a condition-based approach, supporting early fault detection and reducing the risk of unexpected downtime. In future development, additional features such as RPM sensing, fuel-level monitoring, oil pressure monitoring, vibration sensing, mobile-app notifications, and machine-learning-based fault prediction can be incorporated.